Regression Model Quantization Framework Final Report

Gil Benezer
Masters of Science in Artificial Intelligence
Khoury College of Computer Science
Northeastern University
Boston, MA
benezer.gi@northeastern.edu

Brent Garey
Masters of Science in Artificial Intelligence
Khoury College of Computer Science
Northeastern University
Boston, MA
garey.b@northeastern.edu

Ryon Sajnovsky
Masters of Science in Artificial Intelligence
Khoury College of Computer Science
Northeastern University
Boston, MA
sajnovsky.r@northeastern.edu

Abstract

Neural network quantization, converting model parameters from higher to lower numerical precision, is a key technique for reducing memory footprint and computational cost during training and inference (1; 2; 3). However, the vast majority of research focuses on large language models and diffusion architectures (4; 5; 6; 7), and either specifically target LLMs (8) or evaluate performance using image classification (9). Regression models remain critical for scientific modeling and neural state estimation in control applications including autonomous vehicles, robotics, and manufacturing systems (10). We developed a prototype library for systematic evaluation of regression neural network performance under quantization along three axes: prediction error, memory footprint, and inference latency. Users can supply a PyTorch model and an evaluation dataset to evaluate the following: mean absolute error, inference latency percentiles, and model size (both absolute values and relative to baseline) across multiple precision levels and quantization approaches. Post-Training Quantization (PTQ) was implemented using the PyTorch-native TorchAO library (11). To evaluate the PTQ pipeline generally, we trained a multi-layer perceptron on regressing superconductor critical temperature data (12) using an auxiliary pipeline we developed and assessed its performance on CPU and GPU. To further assess utility, we made an additional secondary pipeline to use low-discrepancy sampling to explore the design space of MLPs in this application. We found that ReLU activation functions perform best for this regression task, relative median latency and relative model size are positively correlated, and neither metric have an impact on model error relative to full-precision models. Code for this work can be found at `https://github.com/gbenezer/Quant_Analysis`.

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

1 Introduction

Neural networks are powerful function approximators that are used in myriad technological pipelines for a variety of applications. The requirements that these models have to satisfy are also numerous and often reflect the hardware that they are deployed on. Increasingly, neural network models need to be installed in edge devices with hard limits on memory, power, and numerical precision. Quantization (1; 2; 3) is one important approach for appropriate deployment of neural networks in resource constrained settings like autonomous unmanned vehicle steering or the observation and control of manufacturing systems in air-gapped facilities. Notably, no text or image generation is required for these domains. Most research in neural networks now focuses on generative models and quantizing such models. However, the majority of the aforementioned applications involve regression tasks, which few current tools focus on.

To give more detail, the two basic approaches to model quantization are shown below in Figure 1 from reference (2). PTQ involves the quantization of neural network weights and activations using a small subset of training data to determine quantization parameters through a procedure known as calibration and then quantizing the pre-trained full precision model. The other approach is Quantization Aware Training (QAT), where there is finetuning carried out post-training for improving model performance in the quantized regime.

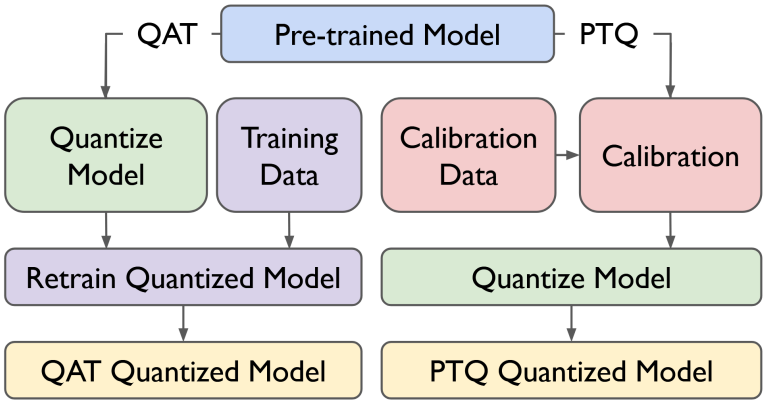

Figure 1: Basic Approaches to Quantization, from (2)

To address this gap, we built a generic pipeline for taking a pre-trained regression model, evaluate a variety of post-training quantization (PTQ) approaches on the pre-trained model, and then summarizing the performance across approaches to inform decision making regarding the deployment of quantized regression models. The main function of the pipeline is to give ML practitioners as well as subject matter experts a sense of the tradeoffs regarding increasing quantization error in order to decrease inference latency and model memory utilization. To evaluate this approach, we also built auxiliary pipelines for training simple feedforward multi-layer perceptron neural networks to predict superconductor critical temperatures as well as sampling uniformly from a predefined feedforward neural network design space.

2 Pipeline Structure

2.1 PyTorch Model Quantization

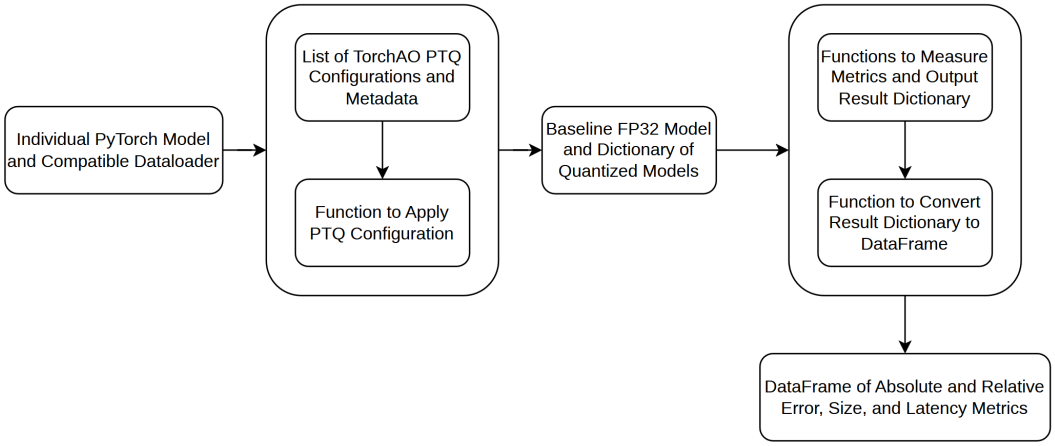

Figure 2: Individual Model Pipeline

Our pipeline, shown in Figure 2, has been streamlined since our proposal submission to only consider post-training quantization (PTQ; quantization of a pretrained model without additional fine-tuning) due to the limited support for quantization-aware training in TorchAO. Right now, we take a 32 bit floating point PyTorch model and PyTorch-compatible DataLoader as input, evaluate baseline metrics (mean absolute error, estimated model size, and multiple inference latency percentiles) on the full precision model, evaluate quantized copies of the baseline model (with caveats), and output a DataFrame with both relative and absolute metric values for each evaluated PTQ approach.

2.2 Multi-Layer Perceptron Construction

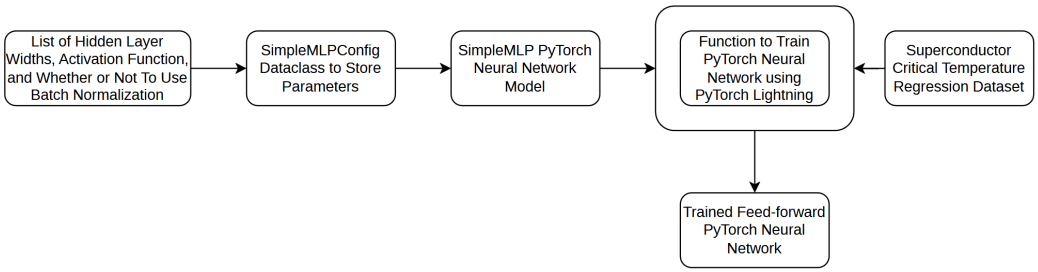

Figure 3: Current Multi-Layer Perceptron Generation Pipeline

We needed a way to test the utility of our quantization pipeline, and so we built an auxiliary pipeline (shown in Figure 3) to generate simple 32 bit floating point multi-layer perceptron PyTorch models trained to predict the temperature at which superconductivity appears in various materials (12). We re-used and updated some of the code from a group member's prior CS 6140 class project (13) utilizing PyTorch and PyTorch Lightning to construct MLP architectures. The main update was to move away from a prescribed design space and set of default configurations for MLP construction and instead define a python dataclass called SimpleMLPConfig that is extensible and can be effectively stored in JSON or YAML format. Using similar underlying code and logic, we can extend the repository with further dataclasses for different model architectures.

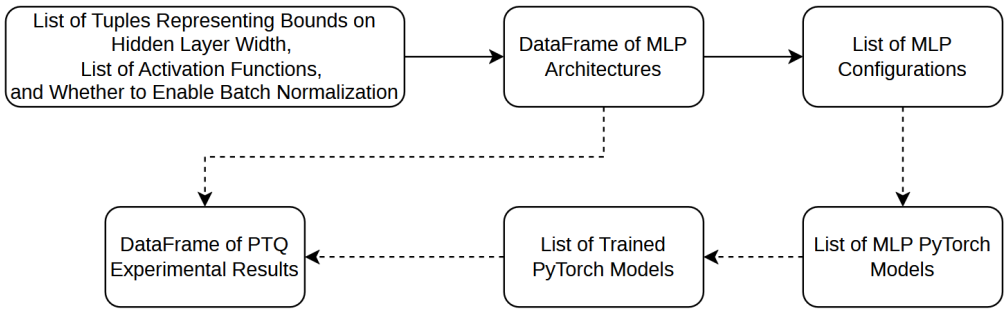

Figure 4: MLP Sampling Pipeline and Ad-Hoc PTQ Integration

2.3 MLP Sampling Pipeline and Integration with PTQ

Finally, to enable exploration of possible MLP architectures, we needed a way to sample from the hybrid continuous (layer widths) and discrete (whether to use batch normalization and choice of activation function) design space. To do this, we constructed a pipeline to generate MLP configurations and a DataFrame describing an experiment from a user specification of the design space (shown in Figure 4). Specifically, we used pseudorandom sampling using low-discrepancy Sobol sequences for choosing layer widths from within the continuous portion of the design space, and we used discrete uniform sampling for choosing activation functions and batch normalization. Due to time constraints, we had to connect the sampling code in an ad-hoc manner to the remainder of the codebase in custom scripts (represented by the dotted arrows versus the solid arrows of the base pipeline), and had to generate custom plots of the resulting data with additional ad-hoc scripts. Future work would focus on connecting these utilities in a more maintainable, sustainable manner.

3 Results

3.1 PTQ Baseline Evaluation

In order to explore the capabilities as well as limitations of our TorchAO-based pipeline, we ran the PTQ pipeline with a standardized 32 bit multi-layer perceptron configuration. In particular, we trained a feedforward neural network model with hidden layer widths [512, 256, 128] and Gaussian Error Linear Unit activation using batch normalization (between the linear and activation layers) on the 81 feature superconductivity dataset (12). We trained a neural network with that configuration using our model pipeline and a custom execution script ten times and used our PTQ evaluation pipeline ten times per training run (for a total of 100 measurements of each metric) on a local CPU, a local NVIDIA GeForce RTX 3070 Laptop GPU, and an NVIDIA H200 graphics card accessed through the Northeastern Research Compute Cluster.

3.1.1 Full PTQ Configuration Comparison

The first assessment we wanted to make was whether there were any differences between local and cluster performance. We saw distributional differences between the two locations in terms of mean absolute error relative to the baseline model, but of low absolute magnitude (1.5%, shown in Figure 5). We also saw slight differences between median latency (shown in Figure 6) between the two locations, but the fact that the data are multimodal suggest that some other dimension is having a larger impact on these metrics.

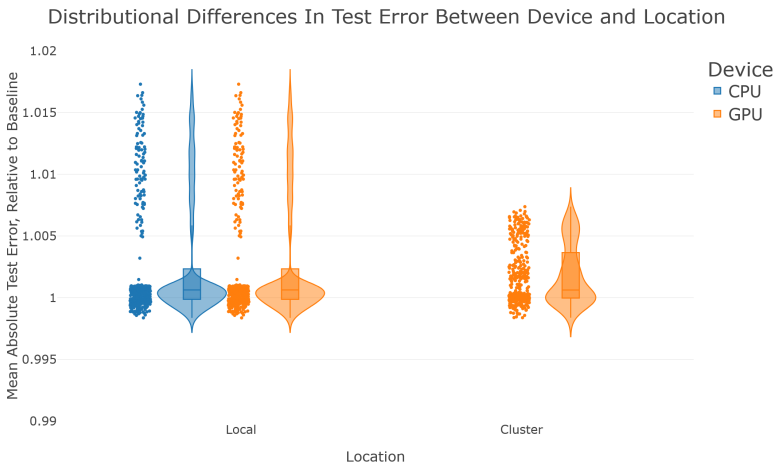

Figure 5: Relative MAE Versus Device and Location

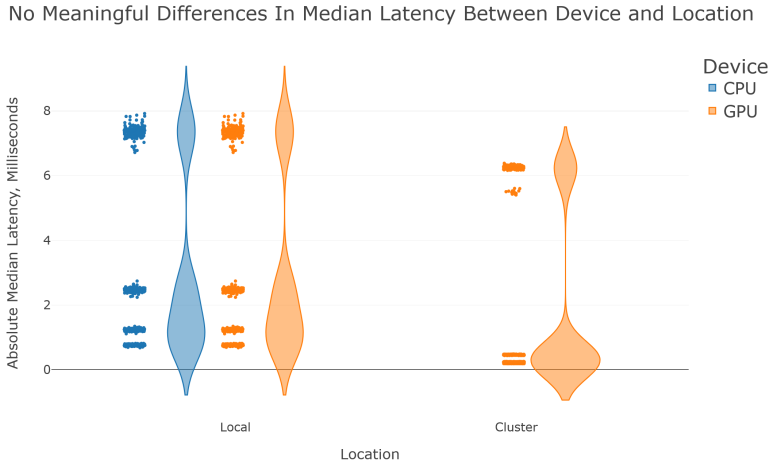

Figure 6: Absolute Median PyTorch Latency by Device and Location

To further investigate this, we looked at the differences between specific tested configurations as a function of computation location, and this is where we started to see clearer distributional trends in Figure 7. In particular, the largest difference was that the only static calibration configuration tested had a higher and wider MAE distribution; this makes sense given that static calibration uses training data to assess calibration parameter values (scale factor, zero point) and is then evaluated on test data. Dynamic calibration approaches calculate calibration parameters dynamically at runtime for decreased error at the cost of increased latency (in theory). However, we dug deeper into the code and this may be a configuration that silently fails on CPU without quantization due to issues with TorchAO and how it is being spun out of PyTorch. The other unexplained difference is that the H200 GPU on the cluster performed worse with Float8 weight only quantization versus local CPU and GPU performance. As a side note, there are no Float8 static calibration results for the cluster evaluation because the CUTLASS kernel used for quantized matrix multiplication on the H200 is not compatible with the input dimension of our data and model (81). All attempts at Int4 configuration testing also failed for similar input dimension incompatibility reasons. There are no Float8 dynamic calibration results because TorchAO requires a GPU with compute capacity above 8.9 to conduct Float8 dynamic calibration.

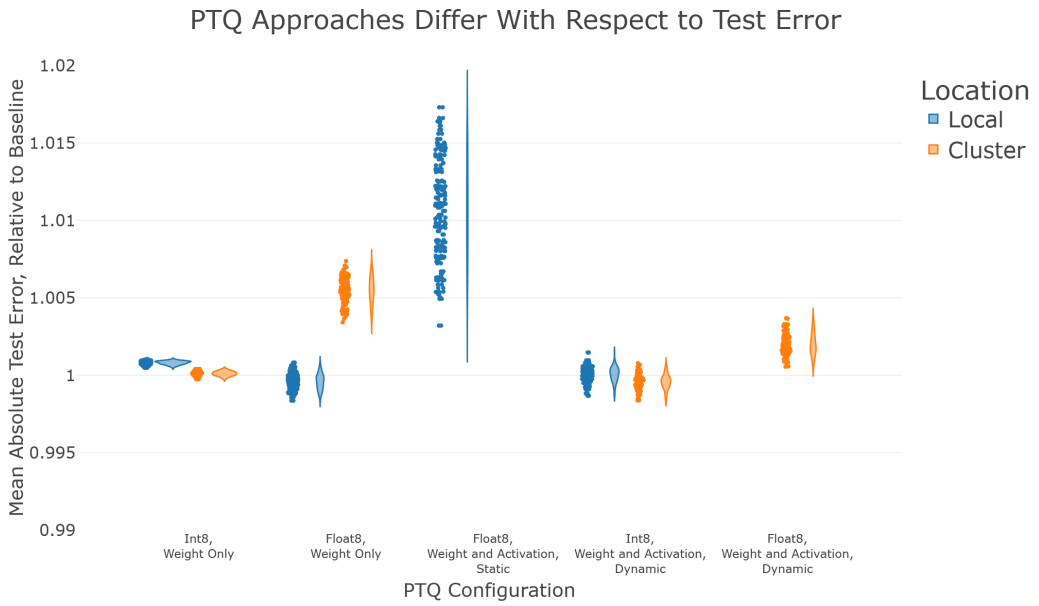

Figure 7: Relative MAE Versus PTQ Configuration and Location

The other important area for comparison across configurations is latency statistics. We also see clear differences in relative median PyTorch inference latency in Figure 8. All of the configurations have relative median latencies above 1, meaning that they are all slower than the base model. This is likely due not to any inherent issues with the quantization, but the fact that inference with a small multi-layer perceptron is very fast on CPU and much faster than that on an H200. As such, the runtime of inference is dominated by quantization overhead, not PyTorch inference. The Int8 dynamic calibration is not shown at all because it does not fit on the axes; the quantized model is about 13 times slower on our local machine and about 63 times slower on the H200 cluster. This could possibly be related to Int8 dynamic calibration PTQ overhead being particularly high relative to the other configurations, but that evidence is circumstantial at best. Additionally, the multiple modes in the distributions evaluated on the cluster suggest that there is an additional unknown variable affecting these latency results.

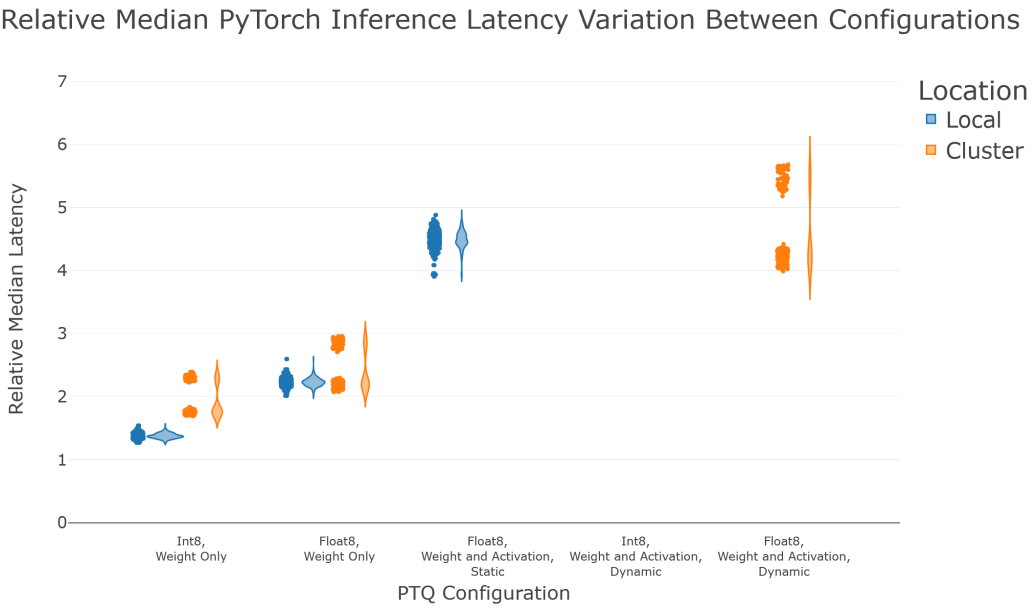

Figure 8: Relative Median Latency Versus PTQ Configuration and Location

3.1.2 Weight-Only PTQ Configurations and Model Size

These latency numbers are dominated by overhead, so we also wanted to evaluate inference as a function of runtime environment. Most TorchAO PTQ quantized models are not compatible with export to ONNX and PT2 file formats, but the weight-only configurations are. Additionally, we had issues with respect to size measurement because of incompatibility of TorchAO PTQ quantized models with export to standard files, so this was the only way we could get measurements of real relative sizes as well. Shown below in Figure 9 is the weight-only configuration runtime comparison results.

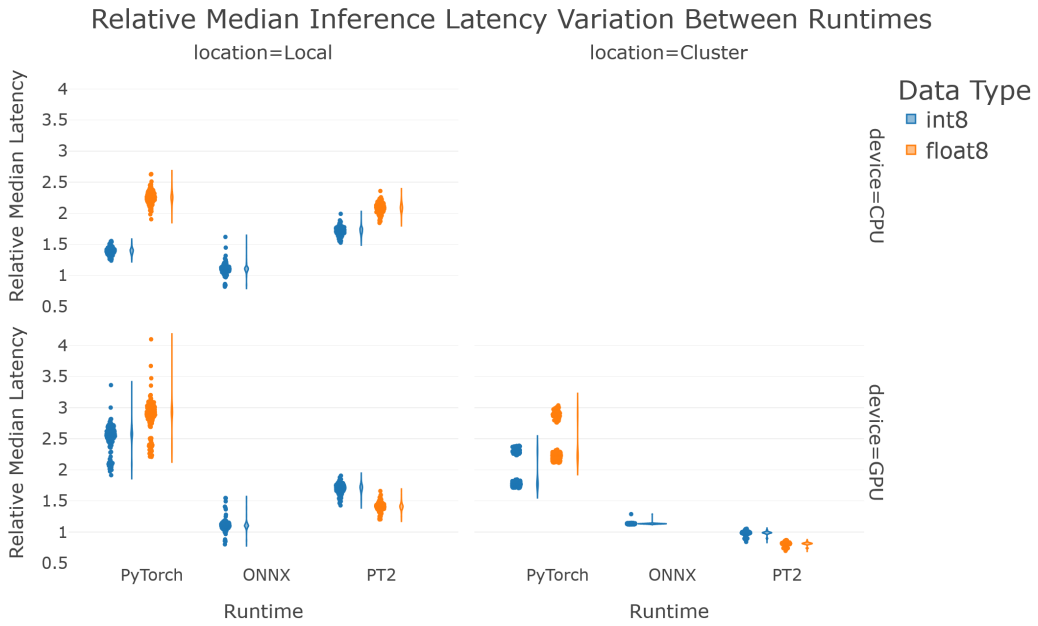

Figure 9: Relative Median Latency, Runtime, and Location

Unsurprisingly, once the models are exported to runtime environments that are not PyTorch, the relative inference latency decreases. In the case of the PT2 runtime, there is actually a relative median latency decrease ($\sim 18\% \pm 5\%$ latency reduction). The more interesting aspect is that most relative median latencies are above 1, meaning that even when exported to different runtimes and compiled the quantized models are slower. The main limitation of these latency results is that they are not executed on deployed edge devices. Another facet of the results is that they show the incompatibility of the Float8 weight only configuration with ONNX runtime export due to lack of support for TorchAO's float8e4m3fn (float 8, 4 bit exponent, 3 bit mantissa, no infinity representation) format.

runtime	data type	median $\pm \sigma$
ONNX	Int8	$1.68 \pm 5.22 \times 10^{-4}$
PT2	Int8	$0.375 \pm 2.72 \times 10^{-4}$
PT2	Float8	$0.400 \pm 4.51 \times 10^{-4}$
PyTorch	8 bit	0.25 ± 0

Table 1: Relative Model Sizes

There's more benefit realized in terms of memory savings, as evidenced by the results in Table 1, but all the model sizes are above the theoretical reduction of 0.25. In the case of the PT2 runtime models, there's minimal overhead and still a savings from storing zero points, scaling factors, and other quantization parameters. In the case of ONNX export files, the exported quantized model is actually larger, but given the limitations of TorchAO it's not clear if this is a real overhead or an artifact.

3.2 MLP Design Space Exploration

To test the newer code allowing for the exploration of the design space (primarily along the dimension of feedforward neural network layer widths and activation functions), we made a custom script to train neural networks with V100 GPUs on the Northeastern Research Compute Cluster. We looked at the design space of three hidden layer feedforward neural networks with a minimum of [128, 64, 32] neurons and a maximum of [1024, 512, 256] neurons. We looked at the rectified linear unit (ReLU), leaky ReLU, exponential linear unit (ELU), Gaussian error linear unit (GELU), and Continuously Differentiable ELU (CELU) activation functions at the same time. The script itself is set to sample 64 feedforward neural network configurations by training each one and using the PTQ pipeline to evaluate the trained models three times. The script was submitted to be executed on the cluster 10 times, for a total of 3 measurements of 640 feedforward neural network configurations. Due to issues with GPUs idling for too long on each submission (leading to early job termination), 281 configurations were actually tested. Additionally, the only TorchAO PTQ configurations that were both agnostic to the dimensionality of the input and hidden layers as well as being compatible with the compute capacity of the V100s were the two weight only configurations (Int8 and FP8) along with the Int8 dynamic activation. As such, this portion of the results section will focus only on the weight-only configurations.

3.2.1 Sample Distribution

The issues with the cluster not fully completing the submitted jobs did cause concern for the distribution of the tested neural network configurations. To get an intuitive sense of the design space and distribution of tested architectures, we first looked at the distribution of tested activation functions; we tested 54 CELU networks, 49 ELU networks, 60 GELU networks, 58 ReLU networks, and 60 Leaky ReLU networks, which is relatively uniform across the 281 architectures. To look at the distribution of hidden layer widths, we visually inspected scatter plots of different layer widths against each other to look for patterns. As can be shown in Figure 10, there are no clear patterns in the distribution of tested layer widths, suggesting that the design space was explored uniformly.

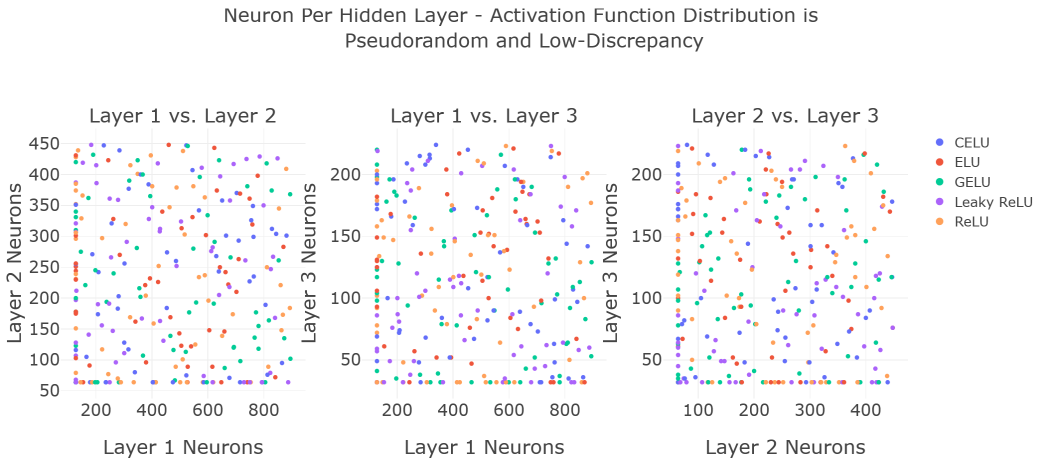

Figure 10: Hidden Layer Width Distribution

3.2.2 Trends in Metrics Versus Architectural Details

Now that we established our dataset is not biased in any particular direction with respect to experimental conditions, we then felt comfortable with moving forward to analysis of the experimental data. As mentioned before, the data examined is for the weight-only configurations. We further focused on the PT2 results for size and latency due to both the compatibility of PT2 with Int8 and FP8 weight-only configurations as well as the compression results from the baseline evaluation making more sense as compared to ONNX exported models.

Given that the most important facet of neural network performance is the model error, the first metric we examined is the mean absolute error of the quantized models. Surprisingly, the only clear trend is in the absolute mean absolute error as a function of activation function, shown in Figure 11.

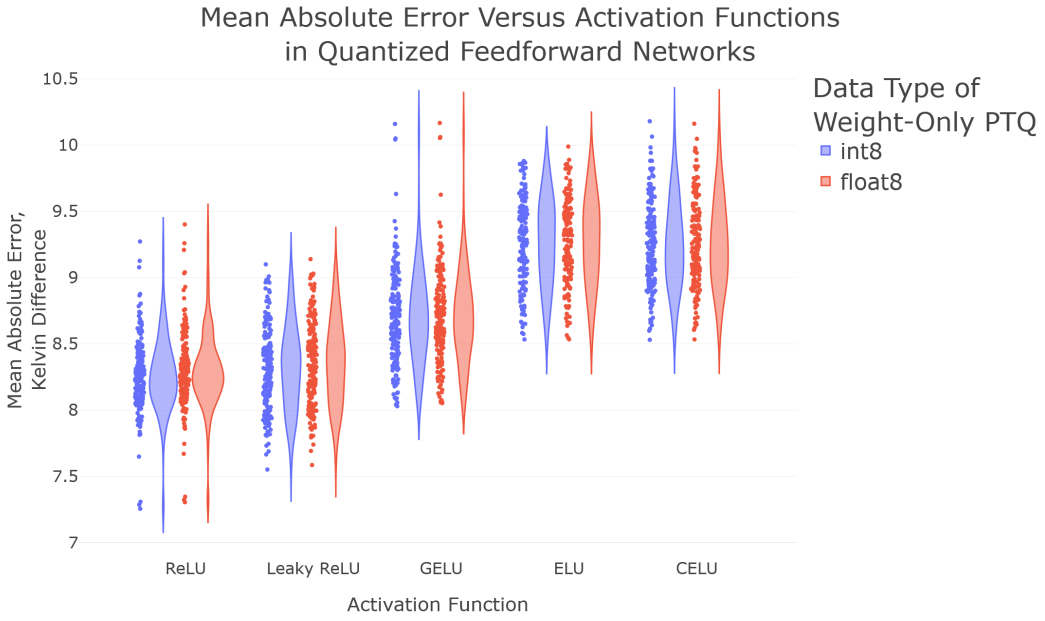

Figure 11: Absolute MAE versus Activation Function

This figure clearly shows that, on an absolute basis, the ReLU function performs better for neural networks trained to predict the superconductor critical temperature (in kelvin). This trend in the absolute value disappears when the performance is measured relative to the baseline full-precision model, shown in Figure 12, but a new trend in variance appears. Given that these measurements are conducted within PyTorch in eager mode, it is likely that this is related to differences in implementation between TorchAO Int8 and FP8 quantization.

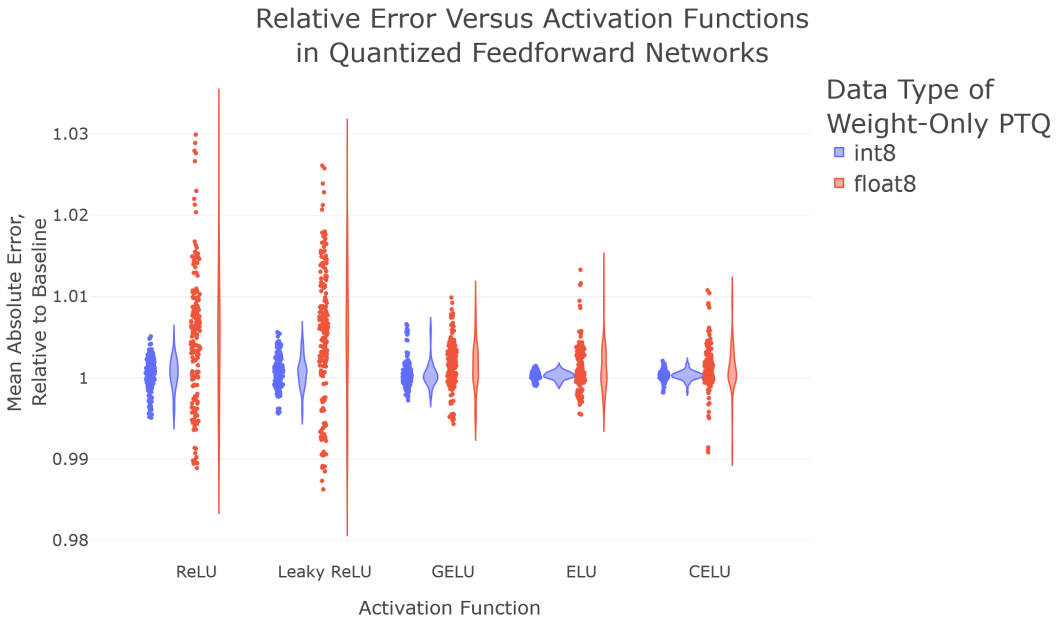

Figure 12: Relative MAE versus Activation Function

Looking next at the median latency of the models (exported to the PT2 file format), there is no clear trend as a function of hidden layer neurons or activation functions. The distribution in median latencies is 0.732 ± 0.042 milliseconds for FP8 weight-only quantization and 0.900 ± 0.049 milliseconds for Int8 quantization. We did, however, see similar phenomena regarding relative latency values as we did in our baseline evaluations; the larger the model is (in terms of hidden layer neurons), the slower the baseline model is relative to the fairly constant speed of inference for the quantized model. This explains the strong negative correlation between model size and relative latency (shown in Figure 13), though not the rationale regarding why FP8 quantization yields faster quantized networks. It also shows the crossover point where it makes sense to quantize the model (between about 800 and 1000 hidden layer neurons).

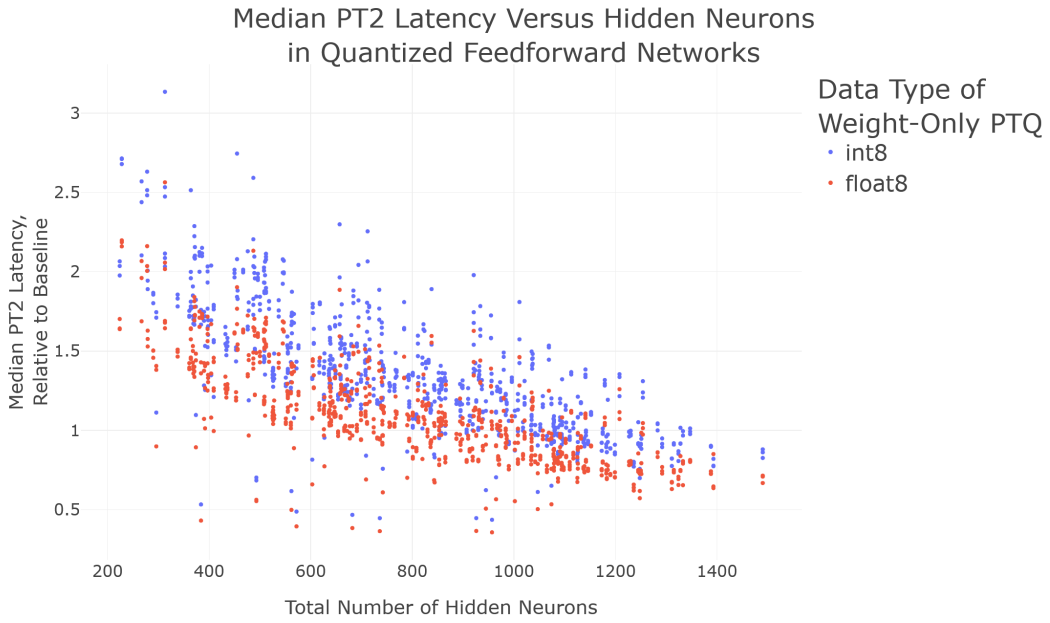

Figure 13: Relative Median Latency versus Total Hidden Layer Neurons

We also checked the trends in model size (from a file size perspective) to be thorough and as a sanity check; we expected to see no trend in absolute model size as a function of activation function and a roughly linear trend regarding total hidden layer neurons. As seen in Figure 14, that is exactly what was observed, though we didn't expect the data to display heteroskedasticity. Interestingly, we also saw a trend in relative model size versus total hidden layer neurons that was similar to the trend in relative median latency (not shown).

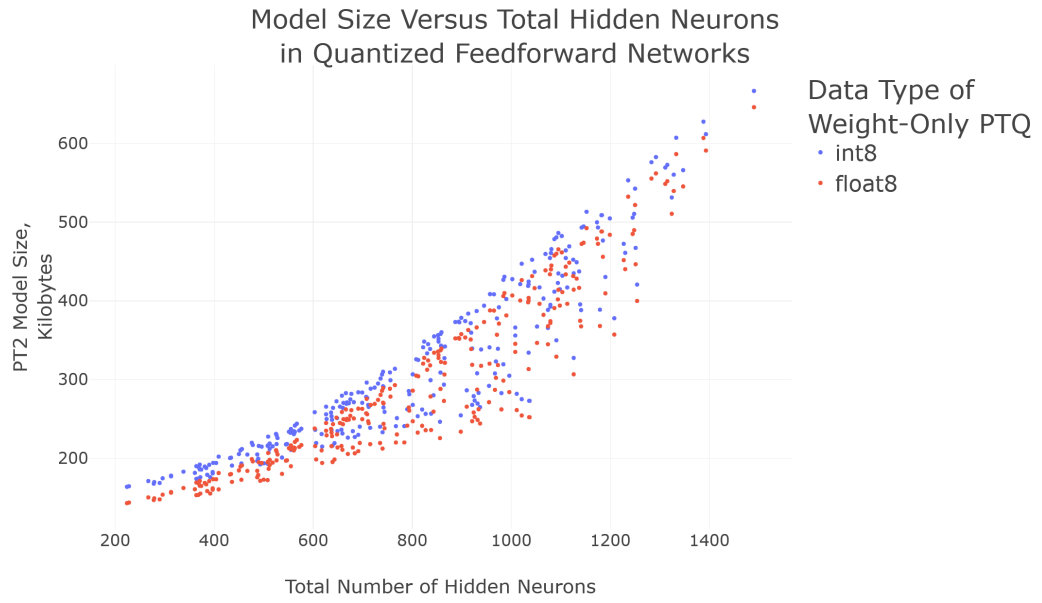

Figure 14: Model File Size versus Total Hidden Layer Neurons

3.2.3 Pareto Frontier and Tradeoff Existence

Ultimately, this project is about developing a pipeline for making decisions regarding neural network architectures as a function of necessary specifications and performance tradeoffs. Therefore, the main results of the experimentation were the construction of Pareto plots showing how performance metrics impact one another. The most interesting finding is that in this design space, the amount of compression achieved and reduction in latency had no impact on model error, implying that there are no tradeoffs here.

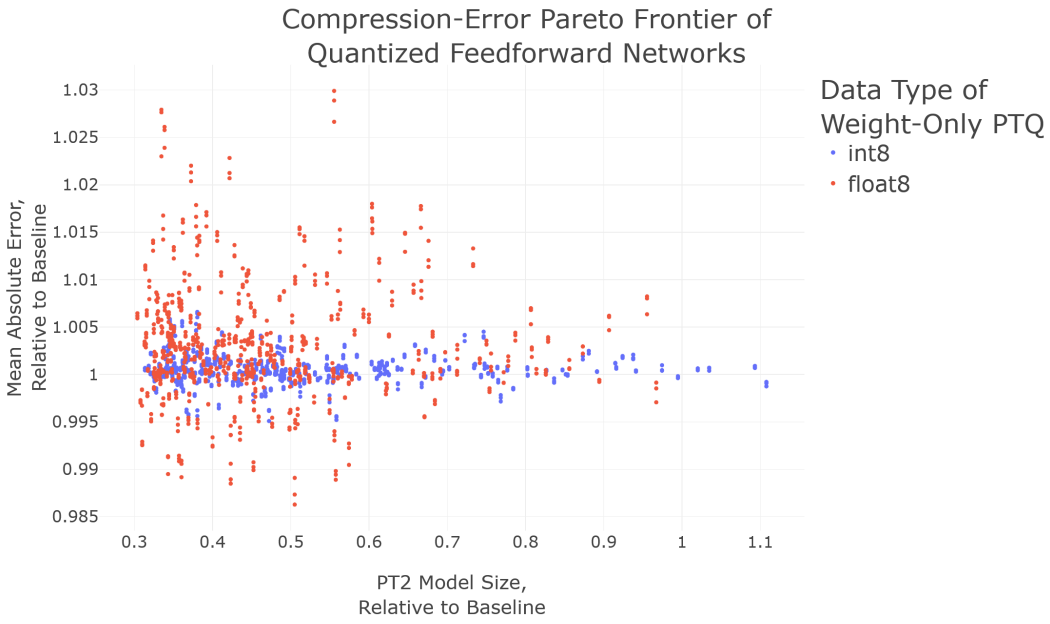

Figure 15: Relative Mean Absolute Error as a Function of Model Compression

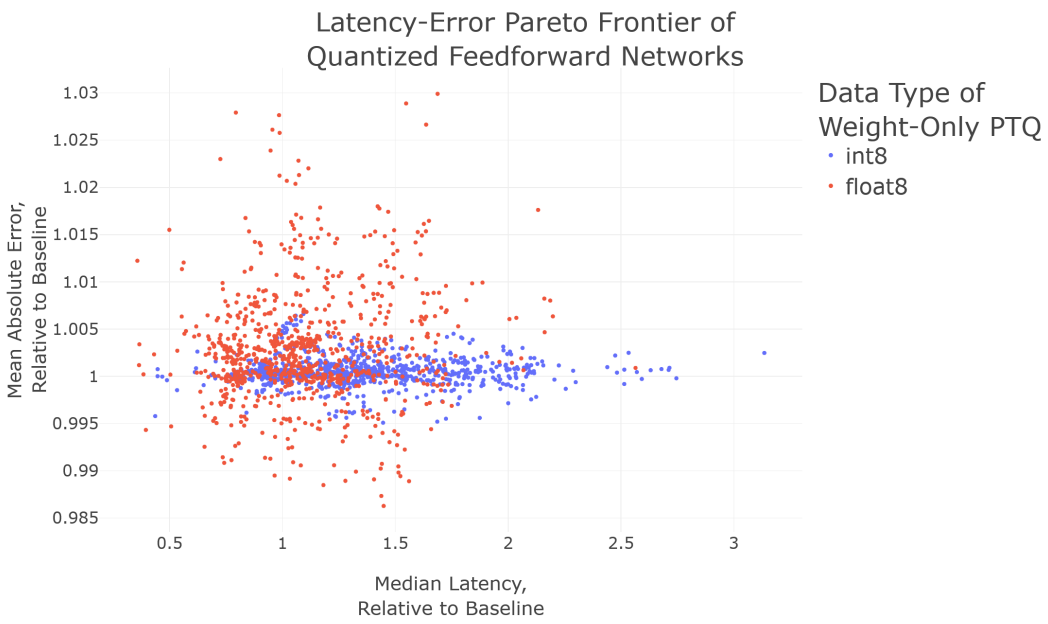

Figure 16: Relative Mean Absolute Error as a Function of Model Latency

Despite the fact that the metrics are secondary in importance, we also looked at the model size - latency Pareto plot for completeness. We found a positive correlation between model size and median latency, shown in Figure 17. Together, all of this suggests that the architecture for predicting superconductor critical temperatures should use ReLU or leaky ReLU activations and the number of hidden neurons should be chosen based on absolute model size constraints.

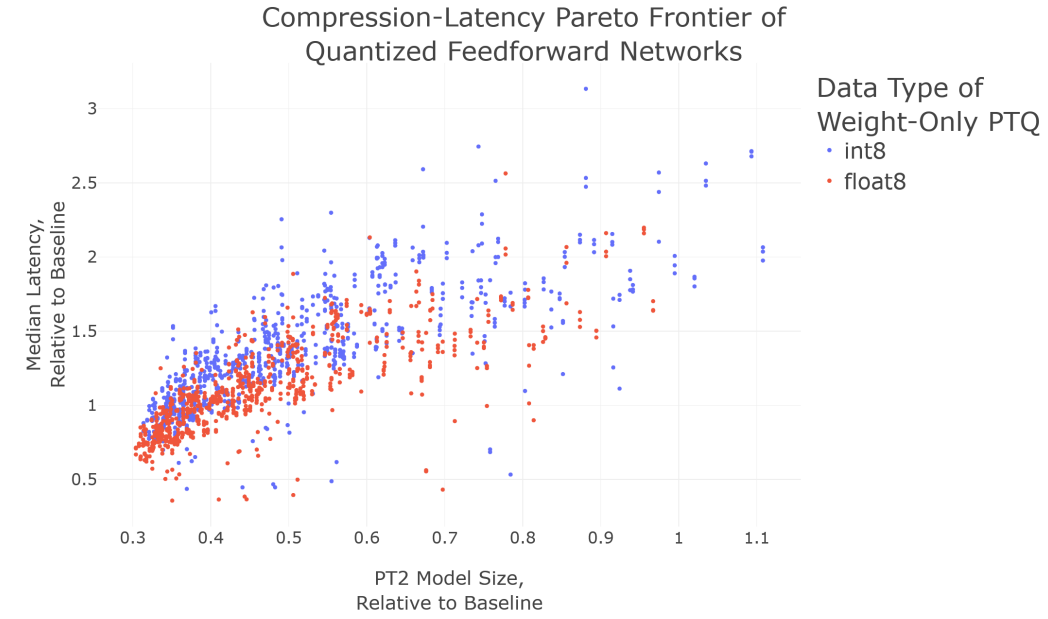

Figure 17: Relative Model Size as a Function of Model Latency

4 Conclusions

We were able to construct a prototype pipeline to take PyTorch neural network models and evaluate their performance after applying post-training quantization (PTQ) along three metrics: model error, size, and latency. We also built auxiliary pipelines to construct and train feedforward multi-layer perceptron neural networks to predict the critical temperature for a material to transition to superconductivity (12) as well as to uniformly sample neural network architectures from a particular design space. Using these pipelines, we were able to learn that weight-only quantization does not have a significant impact on model error, networks with ReLU activations performed the best in terms of absolute error, and that median latency within this design space was effectively constant.

If we were to start the project over again from scratch, the main difference would be that we would not use TorchAO in its current state. TorchAO should not be used for the purpose of quantizing small models. It is not meant for small models, many configurations fail in bizarre ways for non-GPU quantization, the machinery for static quantization in particular is underdeveloped, and the documentation is poor (with multiple broken links on the GitHub project README). This is not surprising in hindsight given that TorchAO is actively being migrated from base PyTorch into its own library. It is also to be expected that small models and embedded systems are not a priority to support, which is one reason why TorchAO is currently incompatible with the on-device PyTorch inference library ExecuTorch (which we wanted to use for latency measurement).

5 Code Repository

The code for this work can be found at `https://github.com/gbenezer/Quant_Analysis`.

References

[1] K. Liu, Q. Zheng, K. Tao, Z. Li, H. Qin, W. Li, Y. Guo, X. Liu, L. Kong, G. Chen, Y. Zhang, and X. Yang, "Low-bit Model Quantization for Deep Neural Networks: A Survey," May 2025. arXiv:2505.05530 [cs].

[2] O. Weng, "Neural Network Quantization for Efficient Inference: A Survey," Jan. 2023. arXiv:2112.06126 [cs].

[3] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A Survey of Quantization Methods for Efficient Neural Network Inference," June 2021. arXiv:2103.13630 [cs].

[4] R. Jin, J. Du, W. Huang, W. Liu, J. Luan, B. Wang, and D. Xiong, "A Comprehensive Evaluation of Quantization Strategies for Large Language Models," in *Findings of the Association for Computational Linguistics: ACL 2024* (L.-W. Ku, A. Martins, and V. Srikumar, eds.), (Bangkok, Thailand), pp. 12186–12215, Association for Computational Linguistics, Aug. 2024.

[5] J. Lang, Z. Guo, and S. Huang, "A Comprehensive Study on Quantization Techniques for Large Language Models," in *2024 4th International Conference on Artificial Intelligence, Robotics, and Communication (ICAIRC)*, pp. 224–231, Dec. 2024.

[6] Q. Zeng, C. Hu, M. Song, and J. Song, "Diffusion Model Quantization: A Review," May 2025. arXiv:2505.05215 [cs].

[7] J. Hasan, "Optimizing Large Language Models through Quantization: A Comparative Analysis of PTQ and QAT Techniques," Nov. 2024. arXiv:2411.06084 [cs].

[8] R. Gong, Y. Yong, S. Gu, Y. Huang, C. Lv, Y. Zhang, D. Tao, and X. Liu, "LLMC: Benchmarking Large Language Model Quantization with a Versatile Compression Toolkit," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track* (F. Dernoncourt, D. Preoţiuc-Pietro, and A. Shimorina, eds.), (Miami, Florida, US), pp. 132–152, Association for Computational Linguistics, Nov. 2024.

[9] Y. Li, M. Shen, J. Ma, Y. Ren, M. Zhao, Q. Zhang, R. Gong, F. Yu, and J. Yan, "MQBench: Towards Reproducible and Deployable Model Quantization Benchmark," Jan. 2022. arXiv:2111.03759 [cs].

[10] Z. Liu, H. Walker, and R. Jain, "Model-Free Neural State Estimation in Nonlinear Dynamical Systems: A Comparative Study of Neural Architectures and Classical Filters," Jan. 2026. arXiv:2601.21266 [cs] version: 1.

[11] t. maintainers and contributors, "torchao: PyTorch native quantization and sparsity for training and inference," Oct. 2024. original-date: 2023-11-03T21:27:36Z.

[12] K. Hamidieh, "Superconductivty Data," 2018.

[13] Gil, "gbenezer/BO-for-NN-HPO-Project," Apr. 2025. original-date: 2025-02-15T23:24:55Z.